ADVANCED PERIPHERAL BUS (APB) IMPLEMENTATION USING VERILOG

PREPARED BY:

Dharmi Patel

Contents

1. ABSTRACT

This project presents the design and implementation of the Advanced Peripheral Bus (APB) protocol using Verilog HDL. APB is a simple and low-power bus protocol that is widely used for communication between processors and low-bandwidth peripheral devices in System-on-Chip (SoC) designs. Due to its simple architecture and minimal control logic, APB is commonly used for connecting peripherals such as UARTs, timers, GPIOs, and communication interfaces.

The proposed design consists of an APB Master and an APB Slave that communicate using standard APB address, control, and data signals. The APB Master is responsible for initiating and controlling read and write transactions, while the APB Slave responds to these requests through an internal memory module. The communication follows the standard APB protocol phases namely Idle, Setup, and Access, ensuring proper synchronization between the Master and Slave.

Additional features such as wait-state insertion using the PREADY signal, error generation using PSLVERR, and support for back-to-back transfers have been incorporated to demonstrate practical APB communication. Various test cases including read operations, write operations, invalid address access, reset conditions, and continuous transfers were verified using a Verilog testbench. Simulation results confirmed the correct operation of the APB protocol and successful data transfer between the APB Master and Slave.

2. OBJECTIVE

- To understand the working of the Advanced Peripheral Bus (APB) protocol.
- To design and implement an APB Master and APB Slave using Verilog HDL.
- To perform read and write operations between the Master and Slave.
- To implement wait-state generation and error handling in APB communication.
- To support back-to-back transfers for continuous data transactions.

3. INTRODUCTION

3.1 INTRODUCTION TO AMBA BUS

AMBA (Advanced Microcontroller Bus Architecture) is an open-standard on-chip communication architecture developed by ARM. It defines how different components such as processors, memory, and peripheral devices communicate within a System-on-Chip (SoC). AMBA provides a standardized and efficient communication framework, making system design easier and more reusable.

The AMBA hierarchy consists of the following protocols:

1. <u>High-Performance Protocols:</u>

- **AXI** (Advanced eXtensible Interface)
  Used for high-speed and high-bandwidth communication between processors and memory.
- **AHB** (Advanced High-performance Bus)
  Used for communication with high-performance peripherals and memory systems.
- **ASB** (Advanced System Bus)
  An older AMBA protocol that has largely been replaced by AHB.

2. <u>Low-Power / Low-Bandwidth Protocols:</u>

- **APB** (Advanced Peripheral Bus)
  A simple and low-power bus used for peripherals such as UART, SPI, I^2C, GPIO, and timers.

3. <u>Coherent and Specialized Protocols:</u>

- **CHI** (Coherent Hub Interface)
  Provides cache-coherent communication in multi-core systems.
- **ACE** (AXI Coherency Extensions)
  Adds cache coherency support to the AXI protocol.
- **AXI-Stream**
  Used for continuous data transfer applications such as video and signal processing.

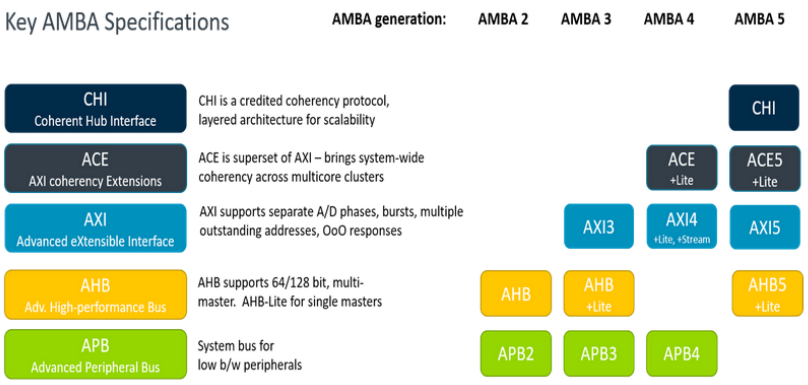

Fig.1: AMBA Protocol Family

3.2 INTRODUCTION TO APB

APB (Advanced Peripheral Bus) is a low-power and low-complexity bus protocol in the AMBA family developed by ARM. It is designed for communication with low-bandwidth peripherals such as UART, SPI, I²C, GPIO, and timers.

Unlike AXI and AHB, APB does not support burst transfers or pipelining, making its architecture simple and easy to implement. APB communication is performed through three phases: Idle, Setup, and Access. It uses a small number of control signals, resulting in reduced hardware complexity and power consumption.

Due to its simplicity and efficiency, APB is widely used for connecting peripheral devices in embedded systems and SoC designs.

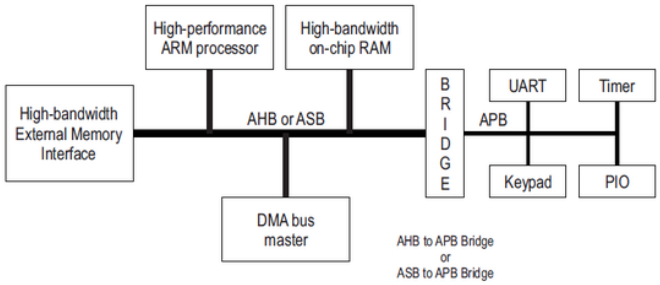

Fig.2: Typical AMBA System Architecture

4. FEATURES OF APB

The Advanced Peripheral Bus (APB) provides a simple and efficient communication interface for low-bandwidth peripheral devices. Its main features are:

- Simple bus architecture with minimal control signals.
- Low power consumption, making it suitable for peripheral devices.
- Supports memory-mapped read and write operations.
- Uses a non-pipelined transfer mechanism.
- Easy to design and implement compared to high-performance buses.
- Suitable for low-speed peripherals such as UART, SPI, I²C, GPIO, and timers.
- Supports wait-state insertion using the PREADY signal.
- Supports error indication through the PSLVERR signal.

5. SIGNALS OF APB

SIGNAL NAME	DIRECTION	WIDTH	DESCRIPTION
PCLK	Global Clock	1 bit	Clock signal used to synchronize APB transfers.
PRESETn	Global Reset	1 bit	Active-low reset signal.
PADDR	Master → Slave	ADDR_WIDTH	Carries the address for read or write operations.
PSEL	Master → Slave	1 bit	Selects the target slave device.
PENABLE	Master → Slave	1 bit	Indicates the Access phase of the transfer.
PWRITE	Master → Slave	1 bit	Determines whether the operation is a read or write.
PWDATA	Master → Slave	DATA_WIDTH	Carries data from the master during write operations.
PRDATA	Slave → Master	DATA_WIDTH	Carries data from the slave during read operations.
PREADY	Slave → Master	1 bit	Indicates that the slave is ready to complete the transfer.
PSLVERR	Slave → Master	1 bit	Indicates an error during the transfer.

*Our design uses ADDR_WIDTH = 32 bits & DATA_WIDTH = 32 bits

6. APB PROTOCOL OPERATION

APB communication takes place through three phases: Idle, Setup, and Access. The APB Master initiates all transfers by providing the address, control signals, and data, while the APB Slave responds to the request. A transfer is completed when the slave asserts the PREADY signal.

6.1 IDLE PHASE

The Idle phase represents the inactive state of the bus. In this phase, no data transfer takes place and the slave is not selected.

- PSEL = 0
- PENABLE = 0
- No transfer is active

6.2 SETUP PHASE

The Setup phase begins when the master initiates a transfer. During this phase, the address, control signals, and write data (for write operations) are placed on the bus.

- PSEL = 1
- PENABLE = 0
- Address and control signals are valid
- Slave is selected

6.3 ACCESS PHASE

The Access phase starts when the master asserts PENABLE. During this phase, the actual data transfer occurs between the master and slave.

- PSEL = 1
- PENABLE = 1
- Read or write operation is performed
- Transfer completes when PREADY = 1
- PSLVERR indicates an error if one occurs

After completion of the transfer, the master either returns to the Idle phase or starts another transfer for back-to-back communication.

6.4 APB TIMING DIAGRAMS

The following timing diagrams illustrate APB read and write transactions with and without wait states.

6.4.1 Write transfer with no wait states

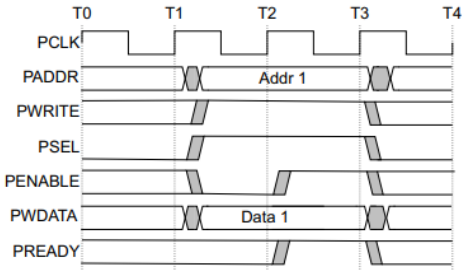

Fig.3: Write transfer with no wait states

1. At T1, the master places the address, write control signal, and write data on the bus.
2. The master asserts **PSEL** to select the slave (Setup phase).
3. At T2, **PENABLE** is asserted, starting the Access phase.
4. The slave asserts **PREADY** immediately.
5. The data is written successfully and the transfer completes at T3.
6. The bus returns to the Idle state.

6.4.2 Write transfer with wait states

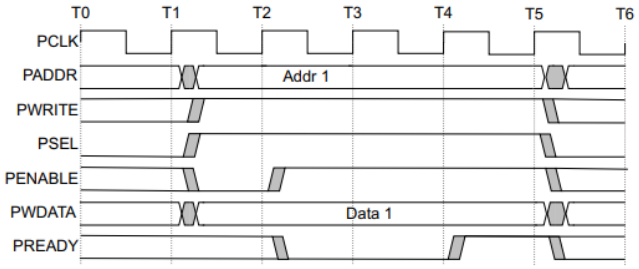

Fig.4: Write transfer with wait states

1. The master starts the transfer by asserting **PSEL** and providing the address and write data.
2. The Access phase begins when **PENABLE** is asserted.
3. The slave keeps **PREADY = 0**, inserting wait states.
4. Address, control, and data signals remain stable during the wait period.
5. When the slave becomes ready, **PREADY** is asserted.
6. The write operation completes and the bus returns to Idle.

6.4.3 Read transfer with no wait states

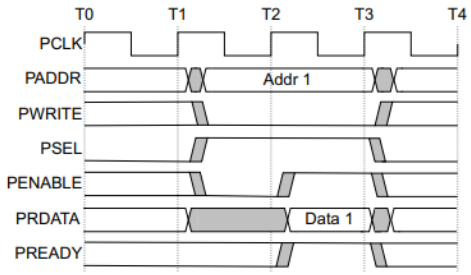

Fig.5: Read transfer with no wait states

1. The master places the address on the bus and asserts **PSEL**.
2. During the Access phase, **PENABLE** is asserted.
3. The slave immediately provides valid **PRDATA**.
4. **PREADY** is asserted by the slave.
5. The master reads the data and the transaction completes.

6.4.4 Read transfer with wait states

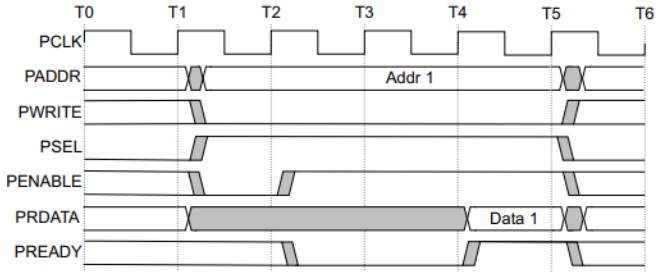

Fig.6: Read transfer with wait states

1. The master initiates a read transaction by asserting **PSEL** and providing the address.
2. The Access phase starts when **PENABLE** is asserted.
3. The slave keeps **PREADY = 0**, delaying the transfer.
4. The address and control signals remain unchanged during the wait states.
5. Once the data is available, the slave drives **PRDATA** and asserts **PREADY**.
6. The master reads the data and the transaction completes.

7. SYSTEM ARCHITECTURE

The proposed APB system consists of an APB Master and an APB Slave connected through standard APB signals. The Master initiates read and write transactions by generating the address, control, and data signals, while the Slave performs the requested operation and returns the response. The design supports wait-state generation using the PREADY signal, error indication through PSLVERR, and back-to-back transfers for continuous communication. An internal memory is implemented within the slave to store and retrieve data during write and read operations.

7.1 OVERALL BLOCK DIAGRAM

The overall architecture consists of an APB Master, an APB Slave, and an internal memory module. The Master receives transfer requests, address information, write data, and read/write control signals from the system. Based on these inputs, it generates the required APB signals such as PADDR, PSEL, PENABLE, PWRITE, and PWDATA.

The APB Slave receives these signals and performs the requested operation on the internal memory. During read operations, data is returned through PRDATA. The slave also generates PREADY to indicate transfer completion and PSLVERR to indicate invalid address accesses. The communication between the Master and Slave follows the standard APB protocol consisting of Idle, Setup, and Access phases.

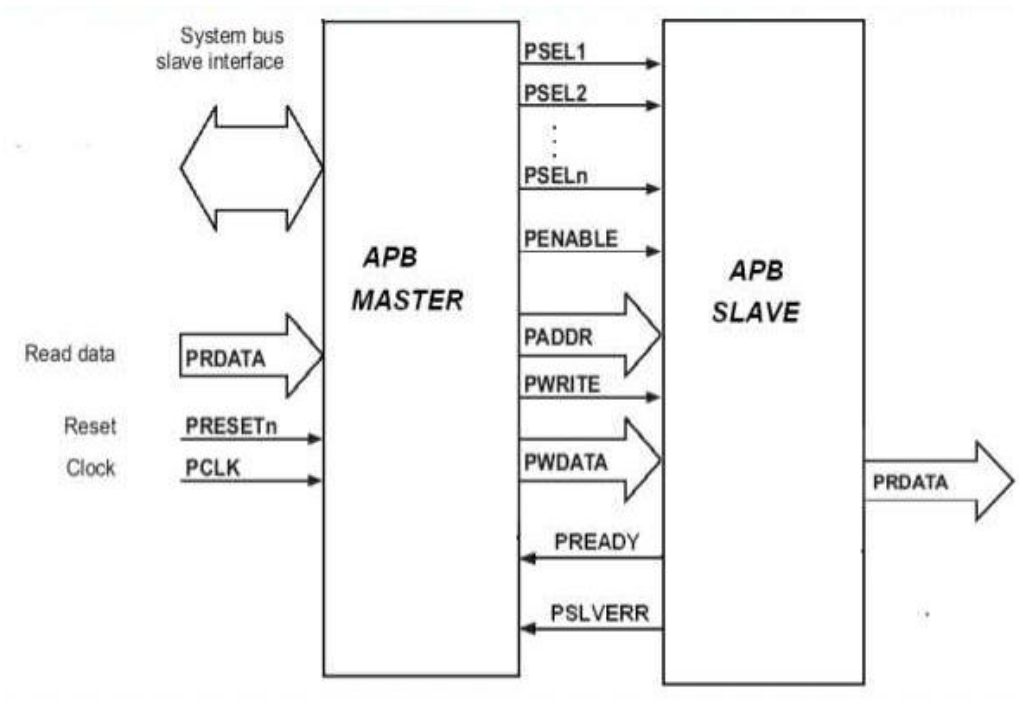

Fig.7: APB Block Diagram

The APB Master and APB Slave communicate through standard APB signals including PADDR, PSEL, PENABLE, PWRITE, PWDATA, PRDATA, PREADY, and PSLVERR.

7.2 APB MASTER DESIGN

The APB Master is responsible for controlling all bus transactions. It receives the transfer request, address, write data, and read/write control signal as inputs and generates the corresponding APB control signals required for communication with the slave.

The Master is implemented using a three-state finite state machine consisting of IDLE, SETUP, and ACCESS states. In the IDLE state, the Master waits for a transfer request. When a transfer request is received, the Master enters the SETUP state and places the address, write control signal, and write data on the bus. In the ACCESS state, PENABLE is asserted and the transfer is completed when the slave asserts PREADY.

For read operations, the Master captures the data available on PRDATA and stores it in the RDATA register. It also monitors the PSLVERR signal and generates an error indication whenever an invalid transaction is detected. The Master supports back-to-back transfers by moving directly from the ACCESS state to the SETUP state when another transfer request is available.

7.3 APB SLAVE DESIGN

The APB Slave responds to the requests generated by the APB Master. It contains an internal memory of eight 32-bit registers that are used to store and retrieve data during write and read operations. The slave receives the address, control signals, and write data from the Master and performs the required operation based on the transfer type.

The Slave is implemented using a three-state finite state machine consisting of IDLE, SETUP, and ACCESS states. During the ACCESS state, the slave checks whether the received address is valid. If the address is valid, the slave performs the requested read or write operation. If an invalid address is detected, the PSLVERR signal is asserted.

To model slower peripheral devices, the slave supports wait-state insertion using a configurable wait counter. The PREADY signal remains low until the specified wait cycles are completed. Once the operation is completed, PREADY is asserted to indicate successful completion of the transfer. The slave also supports continuous communication by allowing back-to-back transfers without returning to the IDLE state.

8. APB FSM

Finite State Machines (FSMs) are used in both the APB Master and APB Slave to control the sequence of operations during a transaction. The implemented design uses three states: IDLE, SETUP, and ACCESS. These states ensure proper synchronization between the Master and Slave during data transfers.

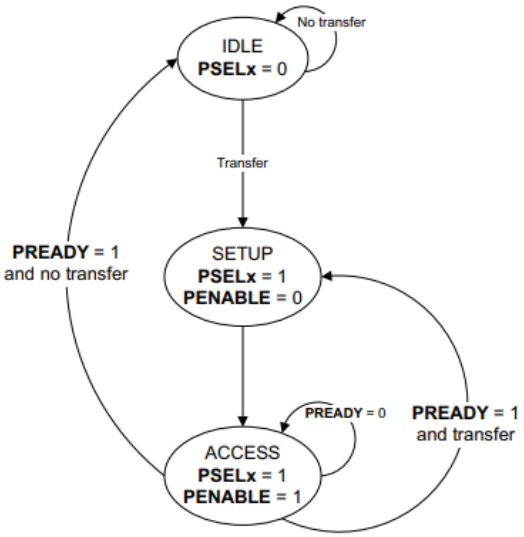

Fig.8: APB FSM

1. **IDLE:** This is the default state of the APB interface.

2. **SETUP:** When a transfer is required, the interface moves into the SETUP state, where the appropriate select signal, PSELx, is asserted. The interface only remains in the SETUP state for one clock cycle and always moves to the ACCESS state on the next rising edge of the clock.

3. **ACCESS:** The enable signal, PENABLE, is asserted in the ACCESS state. The following signals must not change in the transition between SETUP and ACCESS and between cycles in the ACCESS state:
   • PADDR
   • PWRITE
   • PWDATA, only for write transactions
   Exit from the ACCESS state is controlled by the PREADY signal from the Completer:
   • If PREADY is held LOW by the Completer, then the interface remains in the ACCESS state.
   • If PREADY is driven HIGH by the Completer, then the ACCESS state is exited and the bus returns to the IDLE state if no more transfers are required. Alternatively, the bus moves directly to the SETUP state if another transfer follows.

9. WAIT STATE IMPLEMENTATION

In practical systems, peripheral devices may require additional time to complete a read or write operation. To accommodate such devices, the APB protocol provides wait-state support through the PREADY signal.

In the implemented APB Slave, wait states are generated using a configurable wait counter controlled by the parameter WAIT_CYCLES. When the slave enters the ACCESS state, the PREADY signal remains low until the specified number of wait cycles has elapsed. During this period, the Master remains in the ACCESS state and keeps all control, address, and data signals stable.

Once the wait counter reaches the configured value, the slave asserts PREADY, indicating that the requested operation has been completed successfully. The transfer is then completed and the FSM proceeds to the next state. For this design, WAIT_CYCLES is set to 2, resulting in two additional clock cycles before the slave asserts PREADY.

10. BACK-TO-BACK TRANSFER

Back-to-back transfer capability allows multiple APB transactions to be performed consecutively without returning to the IDLE state after each transfer. This improves communication efficiency by reducing unnecessary idle cycles between transactions.

In the implemented APB Master, back-to-back transfers are supported through the state transition logic in the ACCESS state. When a transfer is completed and PREADY is asserted, the Master checks whether another transfer request is available. If a new transfer request exists, the FSM transitions directly from the ACCESS state to the SETUP state, initiating the next transaction immediately.

Similarly, the APB Slave supports continuous communication by transitioning from the ACCESS state to the SETUP state when PREADY is asserted and PSEL remains active. This allows consecutive read or write operations to be performed without returning to the IDLE state.

The implemented feature enables efficient execution of multiple transactions and demonstrates continuous APB communication between the Master and Slave.

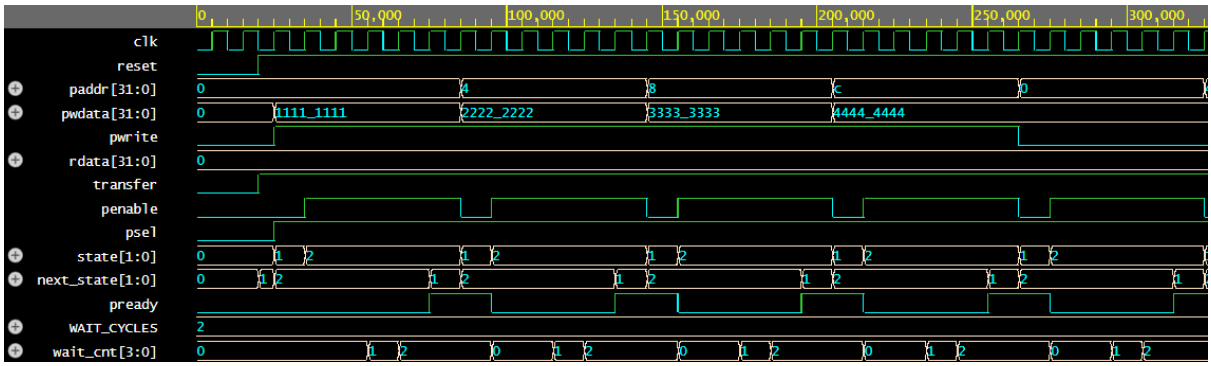

Fig.9: Simulation with wait state count=2 and back to back transfer

11. TESTBENCH DESIGN

A Verilog testbench was developed to verify the functionality of the implemented APB Master and APB Slave. The testbench generates the clock and reset signals, applies various input stimuli, and monitors the outputs of the design under different operating conditions.
The APB Top module was instantiated as the Device Under Test (DUT). Various test scenarios were executed to validate read and write operations, error handling, wait-state generation, reset behaviour, and back-to-back transfers. The testbench also monitors internal signals such as state transitions, PREADY, PSEL, PENABLE, addresses, and data values to verify correct protocol operation.
Simulation results obtained from the testbench confirm the successful operation of the APB communication system.

The following test cases were performed to verify the functionality of the APB design:

1. Write and Read Back Test
   Verifies successful write and read operations.
2. Address Overwrite Test
   Verifies updating data at an already written address.
3. Invalid Address Test
   Verifies PSLVERR generation for invalid memory access.
4. Read Before Write Test
   Verifies default memory contents after reset.
5. Reset During Transaction Test
   Verifies proper recovery when reset occurs during an active transfer.
6. Maximum Address Test
   Verifies access to the highest valid memory location.
7. Multiple Address Access Test
   Verifies read and write operations across multiple memory locations.
8. Back-to-Back Transfer Test
   Verifies continuous APB transactions without returning to the IDLE state.

12. SIMULATION

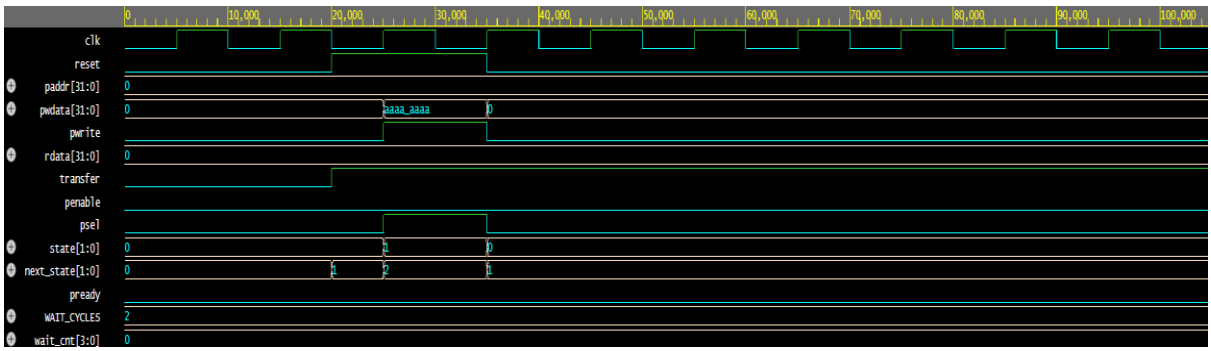

Fig.10: Reset during transmission

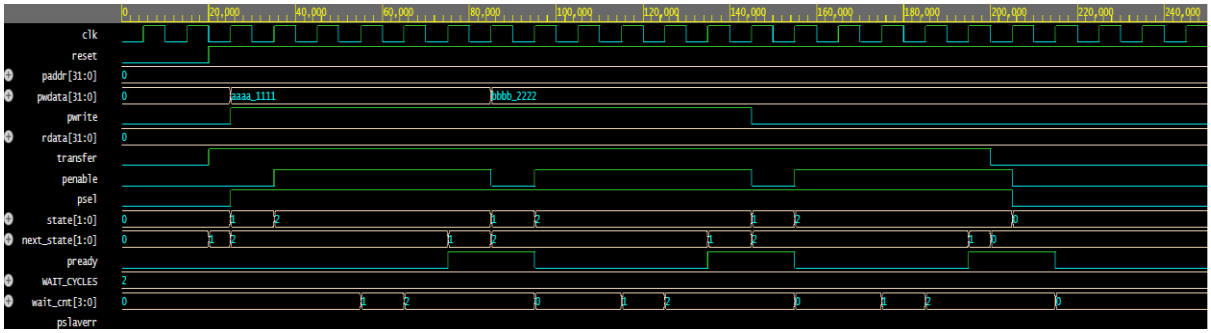

Fig.11: Address rewrite

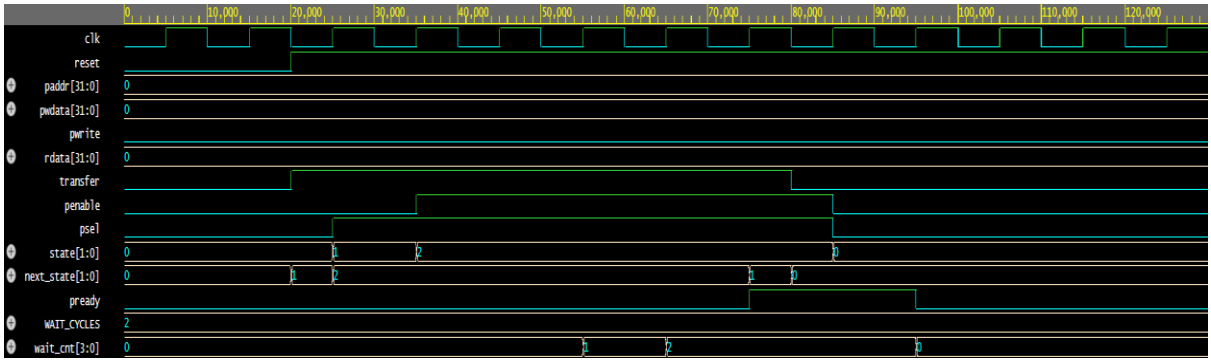

Fig.12: Read before write

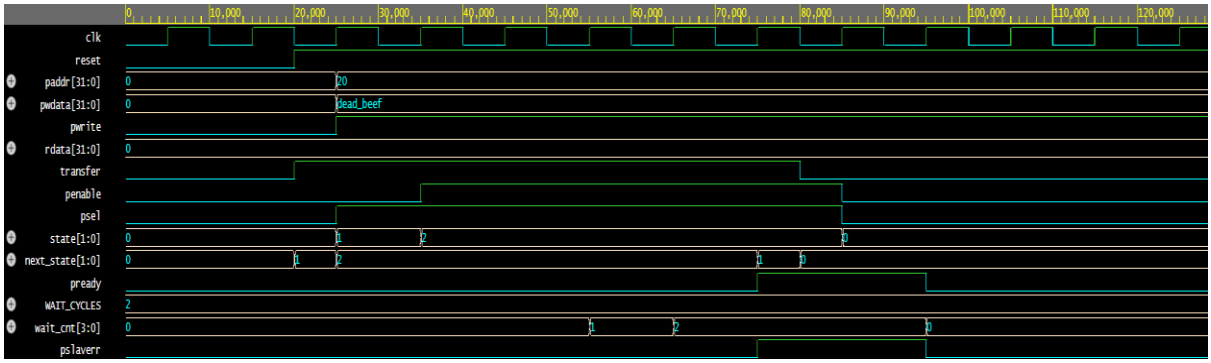

Fig.13: Invalid address

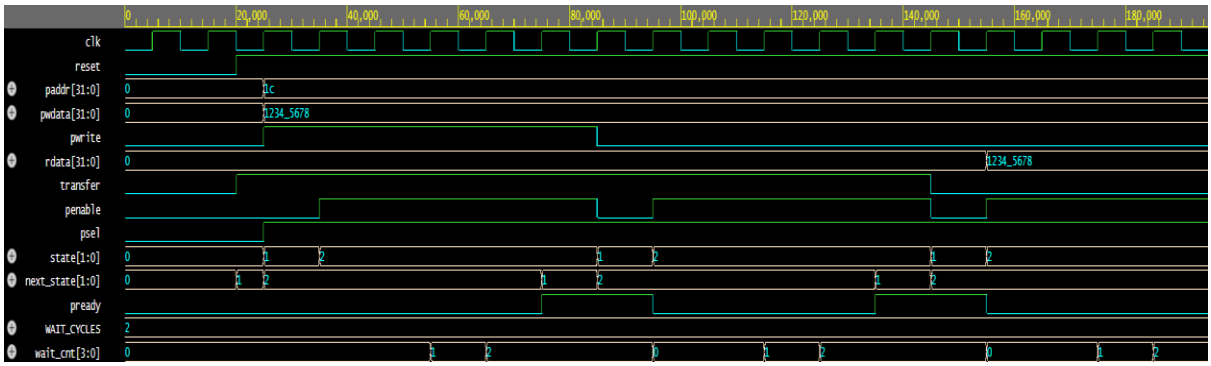

Fig.14: Max location

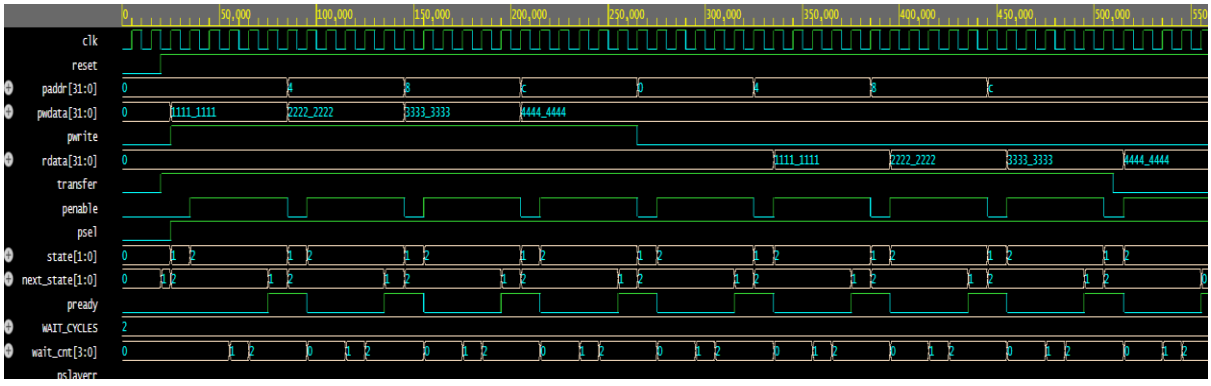

Fig.15: Multiple write and read

13. ADVANTAGES

- Simple and easy-to-understand bus architecture.
- Low power consumption.
- Requires fewer control signals and hardware resources.
- Supports wait-state insertion for slower peripherals.
- Supports back-to-back transfers for efficient communication.

14. LIMITATIONS

- Suitable only for low-bandwidth applications.
- Does not support pipelined transactions.
- Does not support burst transfers.
- Only one transfer can occur at a time.
- Lower performance compared to AHB and AXI protocols.

15. APPLICATIONS

- Communication with peripheral devices such as UART, SPI, I^2C, and GPIO.
- Timer and interrupt controller interfaces in embedded systems.
- Peripheral connectivity in System-on-Chip (SoC) designs.
- Low-power microcontroller-based applications.
- FPGA-based digital system designs and prototyping.

16. CHALLENGE FACED

- Designing the APB Master and Slave state machines according to the APB protocol.
- Implementing wait-state generation using the PREADY signal.
- Handling invalid address accesses and generating PSLVERR correctly.
- Achieving reliable back-to-back transfers without returning to the IDLE state.
- Debugging timing and synchronization issues between the Master and Slave.

17. LEARNING OUTCOMES

- Understanding the architecture and operation of the APB protocol.
- Gaining hands-on experience in Verilog HDL design.
- Learning the implementation of finite state machines (FSMs).
- Understanding wait-state and error-handling mechanisms in bus protocols.
- Developing skills in simulation and verification of digital designs.

18. FUTURE ENHANCEMENT

- Support for multiple APB slave devices.
- Development of an AHB-to-APB or AXI-to-APB bridge.
- Addition of configurable memory size and address space.
- Integration with other AMBA bus protocols.
- FPGA implementation and hardware validation of the design.

19. CONCLUSION

In this project, the Advanced Peripheral Bus (APB) protocol was successfully designed and implemented using Verilog HDL. The design consists of an APB Master and an APB Slave that communicate through standard APB control, address, and data signals. The implemented system supports read and write operations, wait-state insertion using the PREADY signal, error detection through PSLVERR, and back-to-back transfers for continuous communication.

The functionality of the design was verified using various test cases and simulation results, which confirmed correct protocol operation and data transfer between the Master and Slave. This project provided a practical understanding of APB communication, finite state machine design, and digital system implementation using Verilog HDL.

20. REFERENCES

[1] ARM Ltd., *AMBA APB Protocol Specification*, ARM Holdings, Cambridge, United Kingdom.

[2] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*.

[3] M. Morris Mano and Michael D. Ciletti, *Digital Design*.